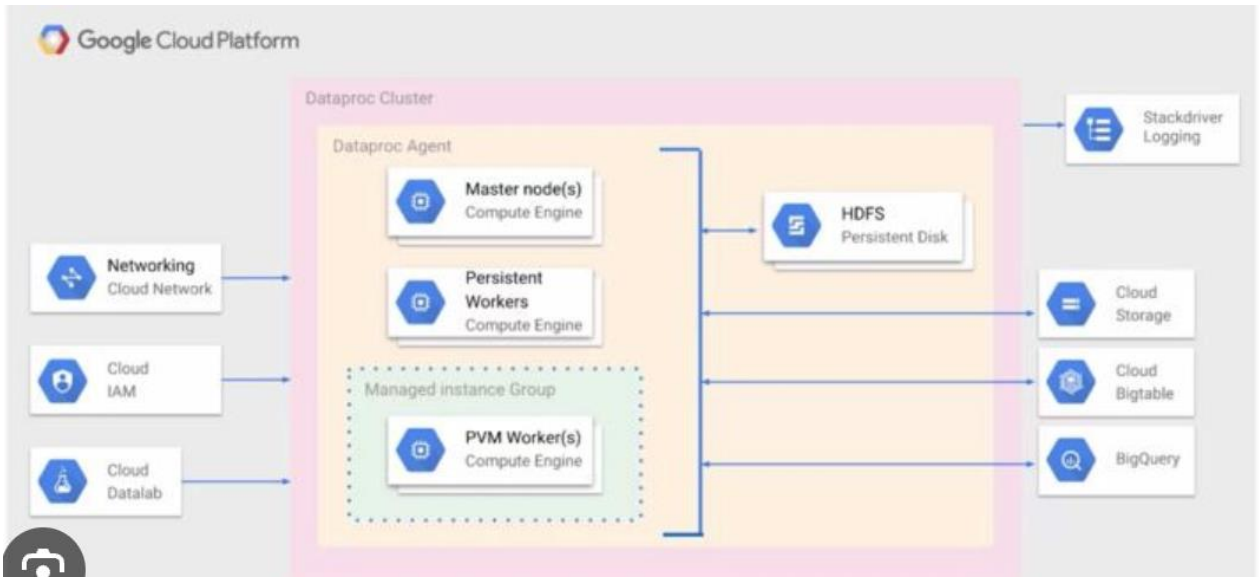

Networking + IAM

|

v

| Dataproc Cluster |

|----------|

| Master Node(s) |

| Persistent Workers |

| Preemptible Workers |

|

+--> HDFS

+--> Cloud Storage

+--> Bigtable

+--> BigQuery

|

+--> Cloud Logging

1. What is Dataproc?

Answer:

Dataproc is a fully managed Google Cloud service for running open-source big data frameworks such as:

- Apache Spark
- Hadoop
- Hive
- Pig
- Presto/Trino

without manually managing clusters.

Typical flow:

GCS → Dataproc (Spark/PySpark/Hive) → BigQuery

2. Dataproc Architecture

Master Node
|
Worker Nodes
|
Spark Executors

Components:

- Cluster
- Master
- Workers
- Spark Driver
- Spark Executors

3. Dataproc v/s dataflow :

We used Dataproc for Spark-based ETL workloads and Dataflow for serverless streaming pipelines.

Dataproc	**Dataflow**
Spark/Hadoop	Apache Beam
Cluster-based	Serverless
Need cluster	No cluster
Batch + Streaming	Batch + Streaming

4. **Dataproc Cluster Types**

Type	**Masters**	**Workers**	**Use Case**
Single Node	**1**	**0**	**Dev/Test**
Standard	**1**	**N**	**Most ETL workloads**
HA	**3**	**N**	**Critical production**

We primarily used Standard Dataproc clusters with one master and multiple worker nodes for PySpark ETL workloads. For cost optimization, clusters were created on demand through Cloud Composer and deleted after job completion.

Can Spark run without workers?- Yes, on a Single Node Cluster.

Why don't we use HA clusters everywhere?

Answer:

HA clusters provide better fault tolerance but are more expensive because they require three master nodes. For most batch ETL workloads, a standard cluster provides sufficient reliability at a lower cost.

Questios:

Q1. How did you use Dataproc?

Sample answer:

At Onix, we used Dataproc clusters to execute PySpark jobs for processing large datasets stored in GCS. The transformed data was written to BigQuery, and jobs were orchestrated through Cloud Composer.

Q2. Why Dataproc?

Answer:

We needed Spark-based distributed processing for large datasets and wanted tight integration with GCS and BigQuery while minimizing infrastructure management.

Q3. Difference between Dataproc and Dataproc Serverless?

Dataproc:

Create cluster
Submit jobs
Delete cluster

Serverless:

Submit Spark job directly
No cluster management

Q4. How do you optimize Dataproc jobs?

Answer:

- Partition data
- Use Parquet instead of CSV
- Tune executor memory
- Reduce shuffle
- Use predicate pushdown
- Autoscaling

Q5. Why use GCS instead of HDFS?

Answer:

Dataproc clusters are temporary, while GCS provides durable and scalable storage. Spark jobs read and write directly from GCS.

6. What is Google Cloud Dataproc and how does it differ from on-prem Hadoop?

Dataproc is a fully managed GCP service for running Spark, Hadoop, Hive, and other big data frameworks. Unlike on-prem Hadoop, Dataproc automatically provisions clusters, handles scaling, monitoring, and integrates with GCP services such as GCS and BigQuery. It reduces operational overhead and allows clusters to be created and deleted on demand.

🔷 How do you create a Dataproc cluster using the gcloud CLI and Terraform?

gcloud dataproc clusters create my-cluster \

--region=us-central1 \

--master-machine-type=n1-standard-4 \

--num-workers=2

Terraform Example:

resource "google_dataproc_cluster" "cluster" {
name = "my-cluster"
region = "us-central1"
}
Interview answer:

In production, we use Terraform for Infrastructure as Code and Cloud Composer or CI/CD pipelines to provision Dataproc clusters automatically.

🔷 How does Dataproc integrate with Cloud Storage, BigQuery, and Pub/Sub?

Storage layer : GCS → Dataproc → BigQuery

BigQuery : Read/write using Spark BigQuery Connector

Pub/Sub : Streaming ingestion source

Example: Pub/Sub → Dataflow → GCS → Dataproc → BigQuery

🔷 What are initialization actions in Dataproc? Can you give real examples?

Scripts executed during cluster creation.

Examples:

Install Python Libraries

pip install pandas
pip install pyarrow

Install JDBC Drivers

wget mysql-connector.jar

Install Custom Monitoring Agents

Example:

install Datadog agent
install Splunk agent

Interview answer:

We used initialization actions to install Python dependencies and JDBC drivers required by Spark jobs.

🔷 How do you automate cluster deletion post job completion?- airflow job
Common Composer workflow:

Create Cluster
↓
Submit Spark Job
↓
Delete Cluster

Airflow Operators:

DataprocCreateClusterOperator
DataprocSubmitJobOperator
DataprocDeleteClusterOperator

Purpose: Reduce cost

🔷 How does autoscaling work in Dataproc and when to use it?
Automatically adds/removes workers based on workload.

Benefits:

Lower cost
Improved performance

Use cases:

Variable ETL workloads
Nightly batch processing

🔷 What are best practices to secure a Dataproc cluster (IAM, VPC, encryption)?

IAM - Principle of least privilege

VPC - Private IP clusters

Encryption - Customer-managed encryption keys (CMEK)

Service Accounts - Dedicated service accounts for Dataproc jobs

Network - No public IPs whenever possible

🔷 How do you run Jupyter notebooks directly on Dataproc clusters?

Enable Jupyter component: --optional-components=JUPYTER

Use cases: Data exploration
Spark development
Model prototyping

🔷 Key cost optimization strategies with Dataproc

Ephemeral Clusters

Create → Run → Delete

Autoscaling

Scale only when needed

Preemptible Workers

70–80% cheaper

Use GCS Instead of HDFS

Persistent storage without running clusters

Serverless Dataproc

Pay per job

🔷 How to troubleshoot a failed Dataproc job using logs/Stackdriver?

Spark UI: Driver logs
Executor logs
Stages
Tasks

Cloud Logging: Formerly Stackdriver

Common Issues: Out of memory
Shuffle failures
Missing dependencies
Permission issues

🔷 Cluster lifecycle – ephemeral vs long-running clusters

Ephemeral

Create
Run job
Delete

Best for batch workloads.

Long-running

Cluster always running

Used for frequent jobs or interactive analytics.

Interview preference:

Ephemeral clusters

🔷 How do you manage dependencies in PySpark/Scala jobs on Dataproc?

Python

--py-files

JAR Files

--jars

Initialization Actions

Install dependencies at startup.

Custom Images

Pre-bake dependencies into cluster images.

🔷 Serverless Spark in Dataproc – What's new and why it matters

No cluster creation.

Submit Spark Job
↓
Google manages infrastructure

Benefits:

Lower operations
Faster startup
Pay per use

🔷 What are some limitations or gotchas of using Dataproc?

Startup Delay

Cluster creation takes time.

Cost

Long-running clusters can be expensive.

Spark Tuning

Still requires Spark expertise.

Preemptible Worker Loss

Workers can be reclaimed anytime.

🔷 Difference between preemptible vs non-preemptible workers

Preemptible

Much cheaper
Can be terminated anytime

Use for:

Worker nodes only

Non-Preemptible

Stable
More expensive

Use for:

Master nodes
Critical workloads

🔷 Monitoring strategies – performance metrics, logs, and alerts

Cloud Monitoring

Metrics:

CPU
Memory

Disk
YARN utilization

Cloud Logging

Spark driver/executor logs

Alerts

Set alerts for:

Job failures
High CPU
Cluster unhealthy

- **Hadoop v/s dataproc :**

Feature	**Hadoop (On-Prem)**	**Dataproc**
Infrastructure	**Self-managed**	**Fully managed by GCP**
Cluster Setup	**Hours/Days**	**Minutes**
Storage	**HDFS**	**GCS + HDFS**
Scaling	**Manual**	**Autoscaling**
Maintenance	**User responsibility**	**Google-managed**
Monitoring	**Manual setup**	**Cloud Monitoring**
Cost	**Always-on hardware**	**Pay-as-you-go**
Integration	**Limited**	**BigQuery, GCS, Pub/Sub, Composer**
Upgrades	**Manual**	**Simplified**

Architecture -

Traditional Hadoop:

Data
↓

HDFS
↓
YARN
↓
MapReduce/Hive/Spark

Dataproc :

GCS
↓
Dataproc Cluster
↓
Spark/Hive/Hadoop
↓
BigQuery

- **If Dataproc already has HDFS, why do companies still use GCS?**

HDFS data is lost when the cluster is deleted, while GCS is persistent, highly available, and much cheaper for long-term storage. Therefore, Dataproc clusters are usually compute-only and GCS acts as the data lake.

- **How do you handle data skew in Dataproc Spark jobs?**

Answer:

Common techniques:

- Repartitioning
- Salting keys
- Broadcast joins
- Filtering early
- Bucketing

Example:

```
large_df.join(
  broadcast(small_df),
  "customer_id"
)
```

>**Why is Spark spilling to disk?**

Answer

Causes:

Insufficient executor memory
Large shuffles
Large joins
Wide transformations

Symptoms:

Slow jobs
High disk I/O
Executor failures

Solution:

Increase executor memory
Reduce shuffle
Optimize joins

- **Why use Parquet instead of CSV?**

Answer

Parquet provides:

Columnar storage
Compression
Predicate pushdown
Less I/O

Typical improvement:

3x–10x faster

- **How do you choose executor memory?**

Answer - Consider:

Dataset size
Shuffle size
Number of cores

Typical tuning:

spark.executor.memory
spark.executor.cores
spark.executor.instances

- **How would you process a 10 TB dataset?**

Answer

1. Store in Parquet
2. Partition data
3. Autoscaling cluster
4. Avoid collect()
5. Use distributed aggregations
6. Use GCS as storage